EKS Pod Best Practices

Solution Architect Recommendations for Spring Boot Microservices on AWS EKS

AWS EKS · Spring Boot · Kubernetes · Production Grade · Security · High Availability · Observability · March 2026 · v1.0

Table of Contents

- 01 · Resource Management
- 02 · Health Checks
- 03 · Pod Disruption Budget
- 04 · Autoscaling (HPA)
- 05 · Security Context
- 06 · Secret Management
- 07 · Network Policy
- 08 · Pod Anti-Affinity
- 09 · Graceful Shutdown
- 10 · Image Best Practices
- 11 · Namespace Isolation
- 12 · Observability
- 13 · Deployment Strategy
- 14 · Full Pod Template
- 15 · Checklist & Priority

1 Resource Management

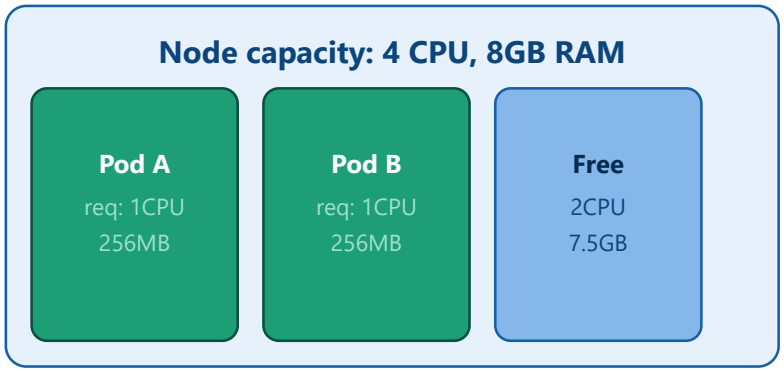
P1 Critical

CPU & Memory requests and limits — the foundation of a stable cluster

⚠️ Missing resource limits is the #1 cause of node crashes and OOMKill in production EKS clusters. Every container must have both requests and limits defined.

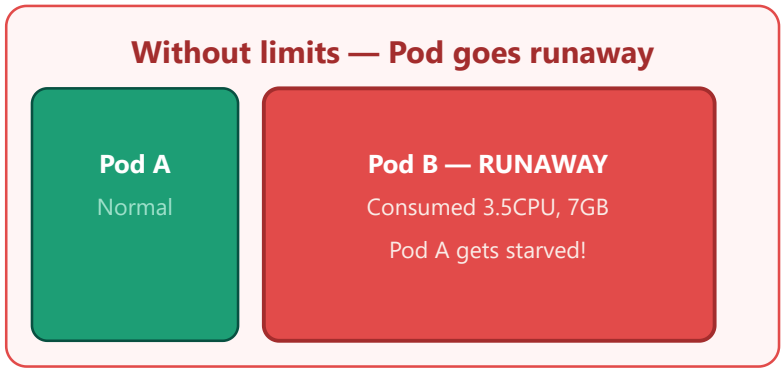
What are Requests?

The **minimum guaranteed** resources the pod needs. Kubernetes uses this to decide which node to schedule the pod on.



What are Limits?

The **maximum** resources a pod can consume. If exceeded, CPU is throttled. If memory is exceeded, pod is OOMKilled.



Correct Configuration

```
resources:
  requests:
    memory: "256Mi"   # guaranteed allocation
    cpu: "250m"       # 250 millicores = 0.25 CPU
  limits:
```

```
memory: "512Mi"   # max — OOMKill if exceeded
cpu: "500m"       # max — throttled if exceeded
```

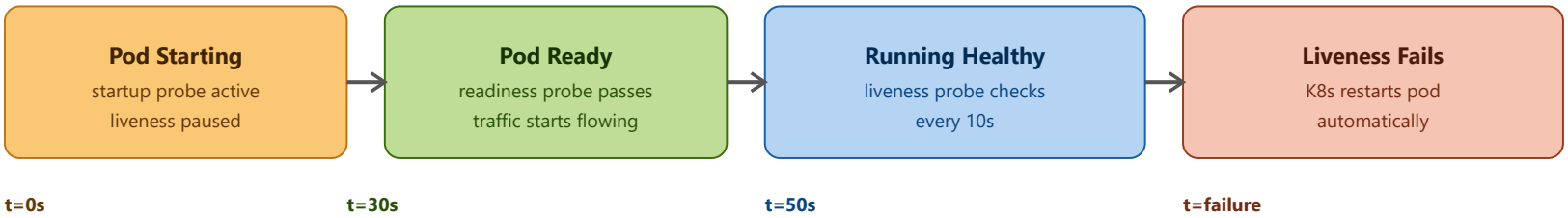
2x	250m	256Mi	OOMKill
limits vs requests starting point	min CPU for Spring Boot pod	min memory for Spring Boot pod	happens when memory limit exceeded

2 Health Checks (Probes)

P1 Critical

Liveness · Readiness · Startup — tell Kubernetes when your pod is healthy

Three Probes — Three Different Jobs



Probe	Question it answers	Failure action	Spring Boot endpoint
Startup	Has the app finished initialising?	Keep waiting (don't restart yet)	/actuator/health
Readiness	Is the app ready to receive traffic?	Remove from load balancer	/actuator/health/readiness
Liveness	Is the app still alive?	Restart the pod	/actuator/health/liveness

Configuration

```
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  initialDelaySeconds: 30   # wait 30s before first check
  periodSeconds: 10          # check every 10s
  failureThreshold: 3        # restart after 3 consecutive failures

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  initialDelaySeconds: 20
  periodSeconds: 5
  failureThreshold: 3        # remove from LB after 3 failures

startupProbe:
  httpGet:
    path: /actuator/health
    port: 8080
  failureThreshold: 30       # allow up to 300s (30 × 10s) to start
  periodSeconds: 10
```

🚨 **Never skip readiness probe.** Without it, Kubernetes sends traffic to your pod the moment it starts — before Spring Boot has loaded beans, connected to DB, or warmed up caches. Users see 503 errors for the first 20–30 seconds of every deployment.

3 Pod Disruption Budget (PDB)

P2 Important

Guarantee minimum availability during node drains, upgrades, and rolling deploys

Without PDB — Risk of Downtime

Rolling deploy without PDB

Pod 1 TERMINATING | Pod 2 TERMINATING | Pod 3 TERMINATING

0 pods running!
DOWNTIME for ~30 seconds

Users see 503 errors

With PDB — Zero Downtime

Rolling deploy with PDB (minAvailable:2)

Pod 1 TERMINATING | Pod 2 RUNNING ✓ | Pod 3 RUNNING ✓

2 pods always running
K8s blocks unsafe drain

Zero downtime guaranteed

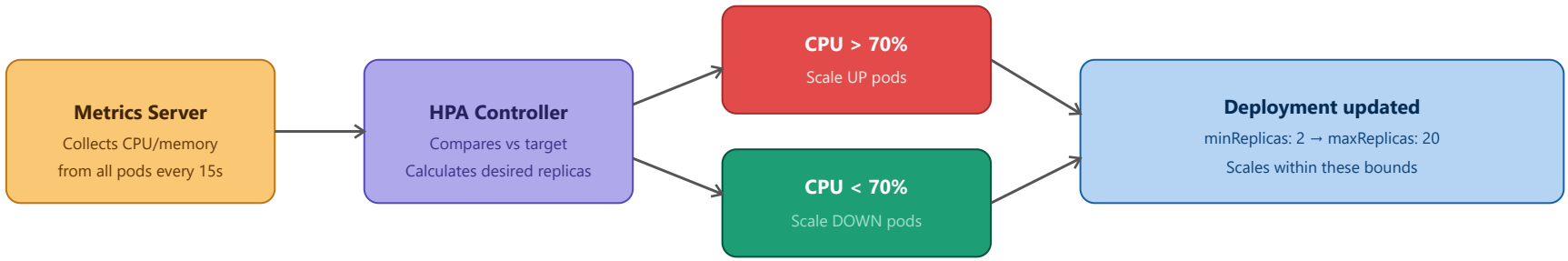
```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: myapp-pdb
  namespace: production
spec:
  minAvailable: 2        # always keep at least 2 pods running
  selector:
    matchLabels:
      app: my-spring-boot-app
```

4 Horizontal Pod Autoscaler (HPA)

P2 Important

Automatically scale pods up and down based on CPU and memory usage

How HPA Works



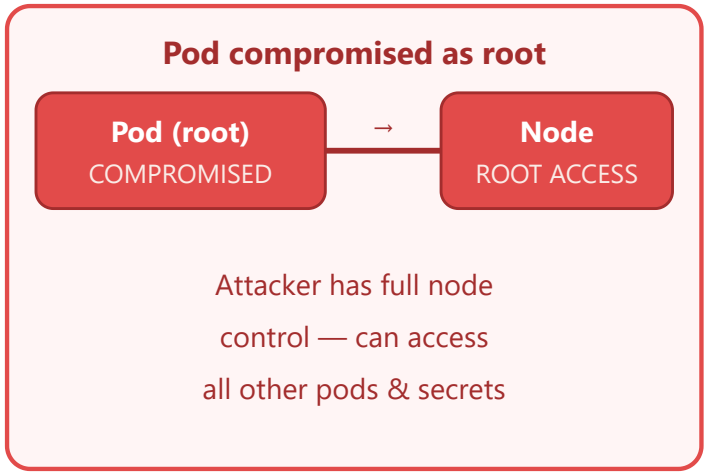
```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: myapp-hpa
  namespace: production
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-spring-boot-app
  minReplicas: 2          # never go below 2 (HA guarantee)
  maxReplicas: 20         # cap to control AWS costs
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70    # scale up when avg CPU > 70%
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80    # scale up when avg memory > 80%
```

5 Security Context — Never Run as Root

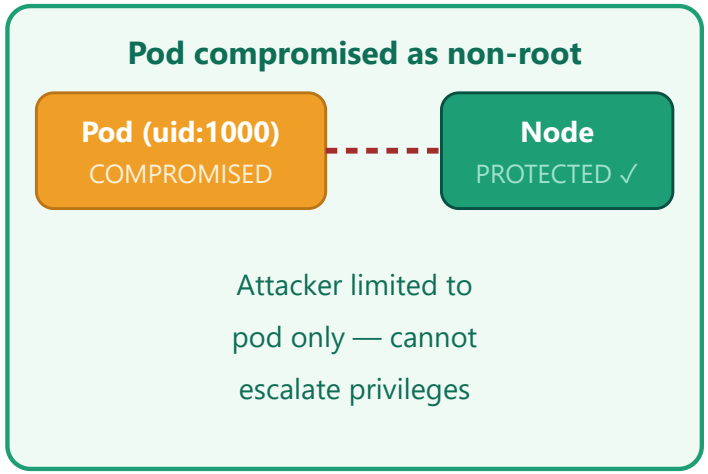
P1 Critical

Contain the blast radius of any pod compromise

✕ RUNNING AS ROOT — DANGER



✅ NON-ROOT — CONTAINED




```
securityContext:
  runAsNonRoot: true              # pod fails to start if image runs as root
  runAsUser: 1000                 # run as specific non-root user
  runAsGroup: 1000
  readOnlyRootFilesystem: true    # filesystem is read-only — attacker can't write
  allowPrivilegeEscalation: false # cannot gain more privileges than parent
  capabilities:
```

```
    drop:
      - ALL                       # drop all Linux capabilities
```

6 Secret Management — Never Hardcode

P1 Critical

Use IRSA + Secrets Manager for zero credential leakage

✕ WRONG — HARDCODED

```
env:
  - name: DB_PASSWORD
    value: "MyP@ssw0rd123"
# Stored in Git forever
# Anyone with repo access
# can see this password
```

✅ CORRECT — SECRETS MANAGER

```
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: aurora-secret
        key: password
# Fetched from Secrets Manager
# Rotates automatically
# Never in Git
```

Credential Flow — IRSA + Secrets Manager

Pod starts IRSA kicks in → AWS validates OIDC token → Secrets Manager returns creds → K8s Secret created → Pod reads via secretKeyRef

```
# Service Account with IRSA annotation
apiVersion: v1
```

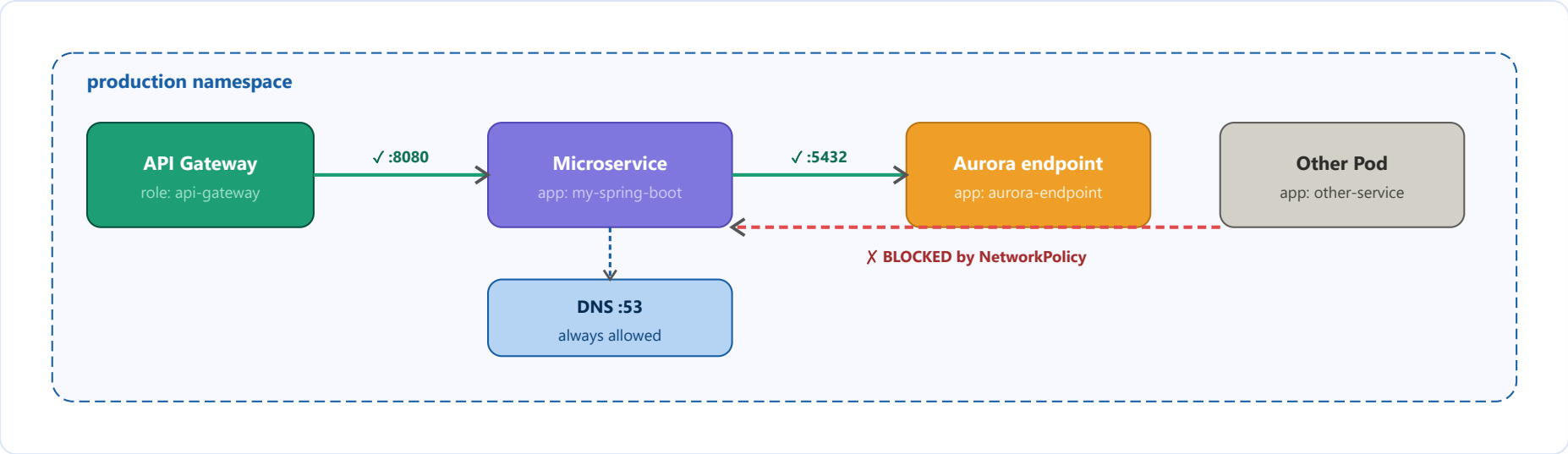
```
kind: ServiceAccount
metadata:
  name: myapp-sa
  namespace: production
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789:role/myapp-aurora-role

# IAM Policy — only allow access to this specific secret
{
  "Effect": "Allow",
  "Action": ["secretsmanager:GetSecretValue"],
  "Resource": "arn:aws:secretsmanager:ap-south-1:123456789:secret:prod/aurora/*"
}
```

7 Network Policy — Zero Trust

P3 Next Sprint

Default deny all traffic, then explicitly allow only what is needed

Zero Trust Network Model




```yaml
# Step 1 — Default deny ALL ingress and egress
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: production
spec:
  podSelector: {}    # applies to all pods in namespace
  policyTypes: [Ingress, Egress]

---
# Step 2 — Allow only what the microservice needs
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: myapp-allow
  namespace: production
spec:
  podSelector:
    matchLabels:
```

```
    app: my-spring-boot-app
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: api-gateway    # only gateway can call me
      ports:
        - port: 8080
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: aurora-endpoint # I can only call Aurora
      ports:
        - port: 5432
    - ports:
        - port: 53                 # always allow DNS
```

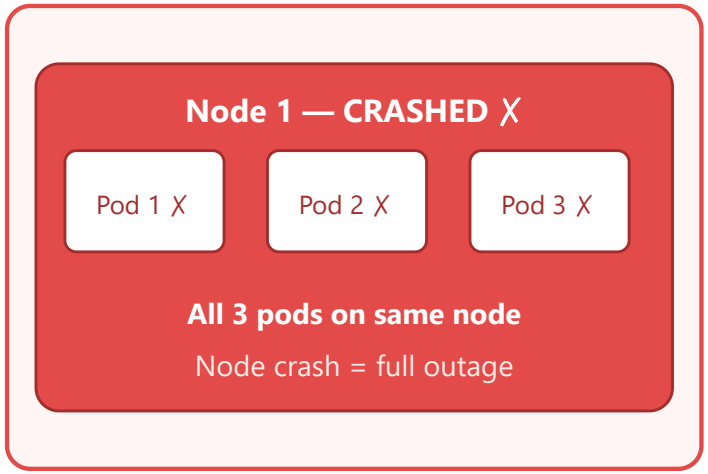
8 Pod Anti-Affinity — Spread Across Nodes

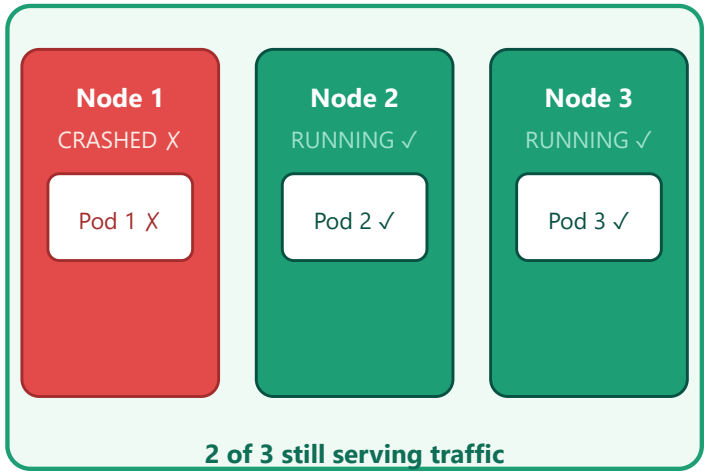
P2 Important

Never put all eggs in one basket — spread pods across nodes and AZs

✕ WITHOUT ANTI-AFFINITY

✅ WITH ANTI-AFFINITY






```
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: app
              operator: In
              values: [my-spring-boot-app]
        topologyKey: kubernetes.io/hostname    # spread across nodes
  # Also spread across Availability Zones
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchLabels:
```

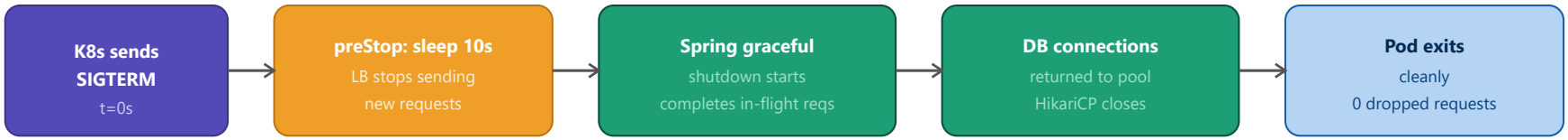
```
            app: my-spring-boot-app
        topologyKey: topology.kubernetes.io/zone  # spread across AZs
```

9 Graceful Shutdown

P2 Important

Drain connections before pod dies — zero dropped requests on every deployment

Shutdown Sequence




```
# Pod spec
terminationGracePeriodSeconds: 60
lifecycle:
  preStop:
    exec:
      command: ["sh", "-c", "sleep 10"]   # wait for LB to deregister

# application.yml
server:
  shutdown: graceful
```

```
spring:
  lifecycle:
    timeout-per-shutdown-phase: 30s       # max time to drain requests
```

10 Image Best Practices

P2 Important

Secure, lean, reproducible container images

Rule	Wrong ❌	Correct ✅
Version tag	**FROM eclipse-temurin:latest**	**FROM eclipse-temurin:17.0.9-jre-alpine**
Base image size	**FROM ubuntu:22.04 (77MB)**	**FROM eclipse-temurin:17-jre-alpine (85MB JRE only)**
User	**No USER directive (runs as root)**	**USER appuser (non-root uid 1000)**
Registry	**docker.io/myapp:latest**	**123456.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0.5**
Scanning	**No vulnerability scan**	**ECR scanning or Trivy in CI pipeline**

```
FROM eclipse-temurin:17.0.9-jre-alpine

RUN addgroup -S appgroup && adduser -S appuser -G appgroup
WORKDIR /app
COPY --chown=appuser:appgroup target/app.jar app.jar
USER appuser

ENTRYPOINT ["java", \
```

```
  "-XX:MaxRAMPercentage=75.0", \
  "-XX:+UseContainerSupport", \
  "-jar", "app.jar"]
```

11 Namespace Isolation

P3 Next Sprint

Separate environments with quotas and RBAC

production
CPU: 10 cores max
Memory: 20GB max
PDB required ✓
Full audit logging ✓
Strict RBAC ✓

staging
CPU: 5 cores max
Memory: 10GB max
PDB optional
Mirrors prod config
Moderate RBAC

development
CPU: 2 cores max
Memory: 4GB max
PDB not needed
Loose quotas
Dev team access

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: production-quota
  namespace: production
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
```

```
    limits.cpu: "20"
    limits.memory: 40Gi
    pods: "50"
    count/deployments.apps: "20"
```

12 Observability

P3 Next Sprint

Prometheus + Grafana + CloudWatch — full visibility into pod health

Observability Stack

Spring Boot /actuator/prometheus → Prometheus scrapes every 15s → Grafana dashboards → Alert Manager threshold breach → PagerDuty / Slack

Critical Alerts to Configure

Metric	Alert Threshold	Meaning	Action
JVM heap usage	**> 85%**	Memory leak risk	Scale up / investigate
HikariCP pool wait	**> 1 second**	DB connection bottleneck	Increase pool size or DB capacity
HTTP 5xx error rate	**> 1%**	Application errors	Check logs immediately
Pod restart count	**> 3 in 5 mins**	CrashLoopBackOff risk	Check liveness probe and logs
CPU throttling	**> 50%**	Limits too low	Increase CPU limits

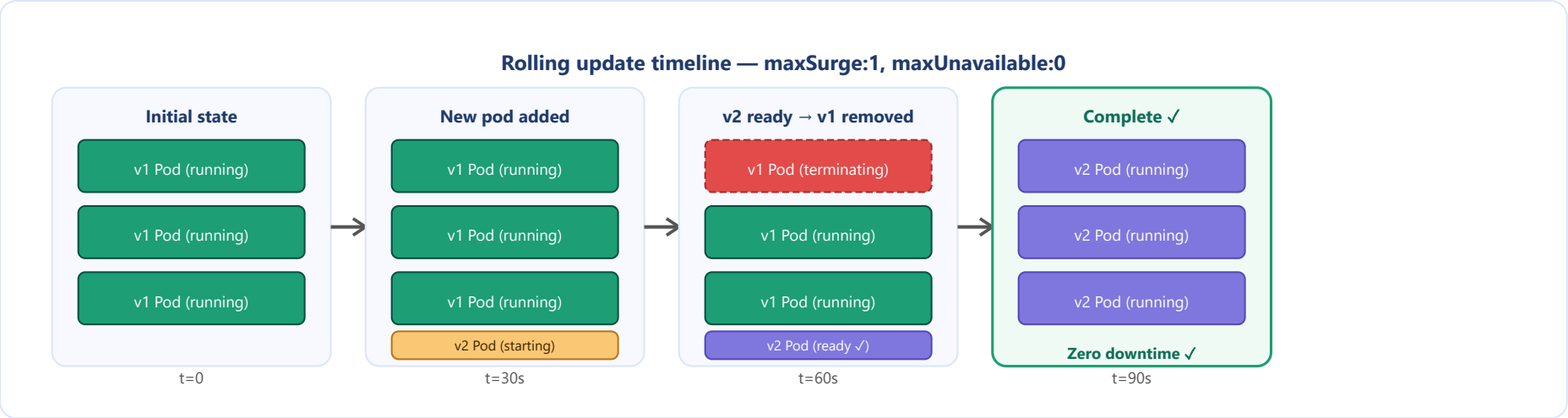
Metric	Alert Threshold	Meaning	Action
Pending pods	**> 0 for 5 mins**	Node capacity issue	Scale node group

```
# application.yml — expose all metrics
management:
  endpoints:
    web:
      exposure:
        include: health, info, prometheus, metrics
  metrics:
    export:
      prometheus:
        enabled: true

# Pod annotations — tells Prometheus to scrape this pod
annotations:
  prometheus.io/scrape: "true"
  prometheus.io/port: "8080"
  prometheus.io/path: "/actuator/prometheus"
```

13 Deployment Strategy

P2 Important

Zero downtime rolling updates with maxUnavailable: 0




```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1        # create 1 extra pod before removing old ones
    maxUnavailable: 0  # never take a pod down before replacement is ready
                       # this combination = zero downtime guaranteed
```

14 Complete Production-Ready Pod Template

Copy-paste ready — all best practices combined into one manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```yaml
  name: my-spring-boot-app
  namespace: production
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: my-spring-boot-app
  template:
    metadata:
      labels:
        app: my-spring-boot-app
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "8080"
        prometheus.io/path: "/actuator/prometheus"
    spec:
      serviceAccountName: myapp-sa           # IRSA role attached here
      terminationGracePeriodSeconds: 60
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchExpressions:
                  - key: app
                    operator: In
                    values: [my-spring-boot-app]
              topologyKey: kubernetes.io/hostname
      containers:
        - name: my-spring-boot-app
          image: 123456789.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0.5
```

```
      ports:
        - containerPort: 8080
      resources:
        requests:
          memory: "256Mi"
          cpu: "250m"
        limits:
          memory: "512Mi"
          cpu: "500m"
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        readOnlyRootFilesystem: true
        allowPrivilegeEscalation: false
        capabilities:
          drop: [ALL]
      env:
        - name: DB_HOST
          valueFrom:
            secretKeyRef:
              name: aurora-secret
              key: host
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: aurora-secret
              key: password
      livenessProbe:
        httpGet:
          path: /actuator/health/liveness
          port: 8080
        initialDelaySeconds: 30
        periodSeconds: 10
        failureThreshold: 3
      readinessProbe:
```

```
      httpGet:
        path: /actuator/health/readiness
        port: 8080
      initialDelaySeconds: 20
      periodSeconds: 5
      failureThreshold: 3
    startupProbe:
      httpGet:
        path: /actuator/health
        port: 8080
      failureThreshold: 30
      periodSeconds: 10
    lifecycle:
      preStop:
        exec:
          command: ["sh", "-c", "sleep 10"]
```

15 Checklist & Priority Order

What to implement and when

Priority Order

Priority	Practice	Risk if Skipped
P1 — Do Today	Resource requests & limits	Node crash, OOMKill cascade
P1 — Do Today	Readiness + Liveness probes	Traffic to unready pods, 503 errors

Priority	Practice	Risk if Skipped
P1 — Do Today	No hardcoded secrets	Credential leak, security breach
P1 — Do Today	runAsNonRoot security context	Full node compromise if pod breached
P2 — This Sprint	PodDisruptionBudget	Downtime during node drain / deploy
P2 — This Sprint	HPA with min 2 replicas	Cannot handle traffic spikes
P2 — This Sprint	Pod anti-affinity	Single node failure = full outage
P2 — This Sprint	Graceful shutdown	Dropped requests on every deploy
P2 — This Sprint	Rolling update maxUnavailable:0	Downtime during deployments
P3 — Next Sprint	NetworkPolicy zero trust	Lateral movement risk
P3 — Next Sprint	Full Prometheus observability	Blind to production issues
P3 — Next Sprint	Namespace resource quotas	Runaway resource consumption

Complete Checklist

- [ ] CPU requests set
- [ ] Memory requests set
- [ ] CPU limits set
- [ ] Memory limits set
- [ ] Liveness probe configured
- [ ] Readiness probe configured
- [ ] Startup probe configured
- [ ] PodDisruptionBudget created
- [ ] HPA minReplicas ≥ 2
- [ ] HPA maxReplicas set
- [ ] Pod anti-affinity configured
- [ ] runAsNonRoot: true

- [ ] readOnlyRootFilesystem: true
- [ ] All capabilities dropped
- [ ] IRSA role attached
- [ ] NetworkPolicy default deny
- [ ] preStop sleep hook
- [ ] Namespace per environment
- [ ] Prometheus metrics exposed
- [ ] Alerts configured
- [ ] Exact version tag pinned
- [ ] allowPrivilegeEscalation: false
- [ ] No hardcoded secrets in YAML
- [ ] Secrets from Secrets Manager
- [ ] terminationGracePeriodSeconds: 60
- [ ] Spring graceful shutdown
- [ ] Resource quotas set
- [ ] Grafana dashboard created
- [ ] Image in ECR
- [ ] Image vulnerability scan

EKS Pod Best Practices · Solution Architecture Team · Version 1.0 · March 2026
AWS EKS · Spring Boot · Kubernetes · Production Grade